

Machine Learning-Based Monitoring Trail: On Fast and Accurate Optical Path Failure Localization in All-Optical DCNs

Zhipeng Zhao¹, Lei Guo¹, Senior Member, IEEE, and Bin Wu²

Abstract—We propose Machine Learning-based Monitoring Trail (mml-trail) to monitor optical path failures for large model training. Existing monitoring trail (m-trail) method can only localize optical link failures, whereas mml-trail differs from it by combining with machine learning and providing bidirectional voltage constraint. mml-trail can provide fast, accurate and integrated failure localization of both optical links and optical switches using only a small number of monitors. Firstly, we construct an input dataset based on the edge relationships of 10000 virtual network topologies (the mappings of optical links and optical switches in all-optical data center networks (DCNs)). Then, monitoring trail under bidirectional voltage constraint is formulated by integer liner program (ILP) to minimize the overall monitoring cost (including monitor and bandwidth costs), and thus construct an output dataset. Finally, we train the learning model based on above dataset using classical and proposed hybrid machine learning models to achieve fast generation of monitoring trails. Based on the constraints of full coverage of monitoring trail and uniqueness of alarms, the generated output results based on machine learning are modified to achieve unambiguous localization for each optical path. Numerical results show that mml-trail outperforms m-trail in localization speed and scalability, and also outperforms machine learning algorithms in accuracy and cost.

Index Terms—All-optical data center networks (DCNs), integer liner program (ILP), optical path failure localization, machine learning-based on monitoring trail (mml-trail).

Received 15 July 2024; revised 6 October 2024; accepted 22 October 2024. Date of publication 24 October 2024; date of current version 3 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62025105 and Grant 62401197, in part by the Natural Science Foundation of Chongqing under Grant CSTB2024NSCQ-MSX0375, in part by China Postdoctoral Science Foundation under Grant 2024MD754042, and in part by the Grain Information Processing Center of Henan University of Technology under Grant KFJJ-2020-111. (Corresponding author: Lei Guo.)

Zhipeng Zhao is with the School of Communications and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China, also with the College of Information Science and Engineering, Henan University of Technology, Zhengzhou 450001, China, also with the Institute of Intelligent Communication and Network Security, Chongqing University of Posts and Telecommunications, Chongqing 400065, China, and also with the Key Laboratory of Grain Information Processing and Control (Henan University of Technology), Minister of Education, Zhengzhou 450001, China.

Lei Guo is with the School of Communications and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China, and also with the Institute of Intelligent Communication and Network Security, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: guolei@cqupt.edu.cn).

Bin Wu is with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin 300072, China.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JLT.2024.3486350>.

Digital Object Identifier 10.1109/JLT.2024.3486350

I. INTRODUCTION

WITH the exponential growth of large model parameters and the advancement of Dense Wavelength Division Multiplexing (DWDM) technology, computing networks such as all-optical data center networks (DCNs) are rapidly developing due to high computing and transportation capacity [1]. In all-optical DCNs for parallel training of large models, optical path flicker caused by dirty optical modules or fiber bending can interrupt the entire training task. Meta once proposed in the training log of the OPT-175B model that almost the entire training process of large model encounters frequent restarts and interruptions, which leads to significant economic losses [2]. According to the 2023 Artificial Intelligence Index report by the Stanford Institute of Artificial Intelligence, OpenAI's GPT-3 consumes 1287 megawatt hours of electricity in a single training session, which means a one-day interruption in power consumption would result in a loss of approximately \$2.5 million [3], raising new challenges in all-optical DCNs management. It is important that the failed optical paths can be immediately localized and bypassed. But it is generally not easy to accurately localize the failures of optical links and switches simultaneously in an instantaneous manner [4]. Due to the fact that the areas of optical propagation and optical switching in the all-optical DCNs do not need to perform photoelectric conversion, traditional electronic methods used to detect failures of CPUs, memory and disks cannot be used to detect optical paths [5].

Current detection method usually equips a monitor for each optical link using a dedicated wavelength as the detection resource, and thus the reserved wavelength cannot be used for traffic transmission. Upon an optical link failure, fast and accurate failure localization can be achieved by collecting alarm codes emitted by the monitor [6]. Although optical link failures can be accurately localized, they require a large number of monitors and assigns detection wavelengths to each optical link, thus consuming a significant amount of network resources. Monitoring cycle (m-cycle) [7] and monitoring trail (m-trail) [8] are proposed to reduce the required number of monitors, but the calculation time of these optimal algorithms is relatively long, which is not scalable for large networks. There are two types of optical switch faults: one is state transition fault, which can be monitored in real-time by on-chip optical power monitoring and other methods to monitor the status of each optical switch unit; Another type is open circuit fault, which can generally

be detected by placing a monitor at the external port of the integrated Optical Cross-connect (OXC). For example, Santec in Japan tests the continuity of the optical path by placing monitor outside the OXC, but can only detect one optical path at a time. As the scale of OXC expands, the detection time and cost increase rapidly.

With the advancement of machine learning technology such as deep learning, more and more researchers begin to focus on intelligent optical network fault prediction. Although current failure detection methods based on machine learning can improve computational efficiency [9], they are still in the prediction and inference stage and cannot accurately locate failures of each optical path. Such methods are suitable for general computing fields but not parallel computing serving large models [10]. In this paper, we propose a Machine Learning-based Monitoring Trail (mlm-trail) method to provide fast, accurate and integrated failure localization of optical links and optical switches by using only a small number of monitors.

Contributions of this paper are summarized as follows:

- 1) We adopt the edge relationships of virtual network topologies (VNT) to construct the input dataset, and the VNT is the mappings of optical links and optical switches in all-optical DCNs.
- 2) We use monitoring trail as the output dataset. Generation of monitoring trails considers bidirectional voltage constraint theory formulated by integer liner program(ILP), and the objective is to minimize the overall monitoring cost including monitor and bandwidth costs.
- 3) We train the learning model based on above dataset using machine learning algorithm to achieve fast generation of monitoring trails. Based on the constraints of full coverage of monitoring trail and uniqueness of alarms, the generated output results based on machine learning are modified to achieve unambiguous localization for each optical path.
- 4) We carry out extensive experiments to demonstrate that the proposed mlm-trail scheme significantly outperforms m-trail in scalability and failure localization speed, and also outperforms machine learning algorithm in accuracy and cost.

The rest of this paper is organized as follow. We review the related work in Section II. The dataset analytics and proposed mlm-trail approach are presented in Section III. We carry out performance study in Section IV before the paper is concluded in Section V.

II. RELATED WORK

A. Optical Link Failure Localization

H. Zeng et al. [11] monitored the link status by constructing a minimum length monitoring cycle (m-cycle) to reduce the detection resources. However, due to the overlap problem of cycles, the m-cycle cannot distinguish the failure of each optical link. H. Zeng et al. [5] also proposed a monitoring tree (m-tree) method, which utilized the broadcasting capability of optical network nodes to achieve accurate failure localization of optical link. However, with the increase in network scale, its scalability is limited. Subsequently, B. Wu et al. [8] proposed a novel fast

optical link failure localization mechanism called monitoring trail (m-trail). M-trail eliminates the cycle constraint of m-cycle and can traverse the same node multiple times. Z. Cheng et al. [12] put forward the traffic trail method, which uses the traffic optical path to replace the specific monitoring optical path, thus saving the localization cost. E. A. Doumith et al. [13] presented a meta-heuristic monitor allocation technique. This method primarily reshapes the trail using the Constrained Trail Reconfiguration (CTR) algorithm, but wastes resources due to its slow convergence speed. For small dense networks, Y. Xuan et al. [14] developed a needle algorithm based on tree decomposition, for multi-link failure localization in sparse networks on a large scale, they proposed a Random Walk (RWL) based localization algorithm. The RWL method locates the failure link more quickly than the CTR algorithm and does not require a complicated trail reshaping procedure. However, the algorithm might not be able to precisely identify the path when there are several identical paths or when the network architecture is complex. C. Ma [15] proposed a Hybrid Localization Integrating Active and Passive (HAP) algorithm that fuses active and passive algorithms. The HAP algorithm pre-processes the alarm information, simplifies the alarm table, and locate the failure location accurately with active monitors. Compared with the RWL algorithm, the HAP algorithm integrates more factors and features, and outperforms than RWL in localization accuracy. However, with a high load and a big alarm set, the HAP algorithm's complexity is considerable. X. L. Wang [16] introduced the Probe Selection (PS) algorithm that locates the failure path after sending detection signals on the defined road and establishing the monitoring path set to be detected first. By optimizing the choice of monitoring path, lowering the average number of wavelengths on each link, and lowering the number of detections necessary, the program lowers resource consumption. The algorithm's practical implementation is limited by the possibility of service disruptions or alterations in huge networks.

B. Machine Learning for Optical Link Failure Localization

Z. Wang et al. [17] studied the relationship between various indexes and the accuracy of fault prediction model, and proposed a novel method based on Double Exponential Smoothing (DES) and Support Vector Machine (SVM). SVM is a combined optical network device failure prediction method, which can back up data and switch services to a secure link before a network failure occurs. However, because it cannot accurately predict the inflection point of the trend, this method does not respond well when the trend changes significantly. Y. Zhao et al. [18] proposed a Self-Optimizing Optical network (SOON) based on machine learning. They combine machine learning techniques with SDON to replace Software-defined Optical networks with self-optimizing network elements (SNE) at the data layer for the first time. Thanks to SNE's ability to process network and business Big Data and extract key features, applications can invoke these features of the control plane to self-optimize critical operations for fault prediction. However, it is difficult to collect network device performance and alarm data due to different device types and versions of different manufacturers.

B. Yan et al. [[19]–[20]] proposed a prediction method based on dirty data and generating adversarial network to overcome the influence of unbalanced distribution in data sets. In addition, C. Y. Zhang [21] developed an intelligent fault analysis method based on cause perception in view of the interpretative problems of fault prediction and analysis models. This method uses the highly interpretable eXtreme Gradient Boosting (XGBoost) technology to build the fault detection and cause analysis model. He also introduced a method for identifying potential failure causes based on attention mechanisms. Through this mechanism, potential failure causes in optical network devices can be revealed based on the calculated attention weights. All these methods are rough estimates of whether faults will occur. In order to quantify the uncertainty of prediction, L. Wang et al. [22] proposed a fault prediction uncertainty analysis technique for optical transmission networks based on Bayesian neural networks. They combine the classical probabilistic model based on Bayesian method with Bayesian Neural Networks (BNN), and solve the problem that posterior probability is not available in Bayesian reasoning by introducing variational reasoning. J. N. Yang [23] designed an innovative fault prediction platform combining LSTM and federated learning. The platform can deeply explore the internal correlation of data to improve the accuracy of prediction, while ensuring the security of data privacy and reducing the cost of model construction. These algorithms mentioned above are all supervised learning algorithms. If unsupervised learning technology is relied on for fault prediction, it is often difficult to achieve high accuracy, and the model trained only through normal data is not accurate in offline fault prediction. In view of this, Z. L. Cheng et al. [24] proposed a new process fault prediction method that integrates CNN, LSTM and Local Outlier Factor (LOF). Combined with CNN-LSTM, this method extracts the features of the fault data set, and uses the trained LOF model to identify the outliers in the predicted values to accurately predict the potential faults. This method can predict the occurrence of faults in advance, and at the same time, it does not alarm early, and has a good prediction effect.

C. Our Work

Our work address the optical path failure localization problem in all-optical DCNs using the method combining monitoring trail and machine learning. We consider two optical path failure types: optical link and optical switch. In previous work, the optical link and optical switch are usually detected separately. In this work, we extend earlier work by considering the integrated detection of optical link and optical switch, which increases the complexity of the problem. Our proposed machine learning based on monitoring trail (mlm-trail) algorithm can achieve accuracy and localize optical path failures faster.

III. DATASET ANALYTICS AND MLM-TRAIL DESIGN

The design process of machine learning-based monitoring trail (mlm-trail) shown in Fig. 1. The dataset is first constructed, where the input data is constructed based on the edge relationships of a virtual network topology composed of optical propagation and optical switching, and the output data is the

monitoring trail result generated through the ILP formulation. Then, based on this dataset, the results generated by machine learning algorithms will be corrected based on the uniqueness constraint of alarm codes. The specific description includes 5 steps as follows:

1) *Original Data*: The all-optical DCNs are first randomly generated by Python tools in the form of fixed nodes and non fixed edges. Then, the optical links and optical switches in all-optical DCNs are mapped as virtual network topologies. The original data includes 10000 virtual network topologies, which is represented in the form of an adjacency matrix G .

2) *ILP Model*: Based on the constraints of voltage legality, direction legality, and alarm code uniqueness, the original data G is input into the ILP model to obtain the monitoring trails with the lowest cost.

3) *Dataset*: The extraction process of the input data includes swapping points and edges to obtain edge relationship data rG , as shown in Algorithm 1. Besides, the monitoring trails obtained by ILP model is reconstructed as the output data Out_{result} , as shown in Algorithm 2. The representation of the dataset C as defined below:

$$C = \begin{bmatrix} rG_{1,1} & Out_{result_{1,1}} \\ rG_{1,2} & Out_{result_{1,2}} \\ \dots & \dots \\ rG_{i,j} & Out_{result_{i,j}} \\ \dots & \dots \\ rG_{J_M,r} & Out_{result_{J_M,r}} \end{bmatrix} \quad (1)$$

where J_M is the number of m-trails and r is the number of optical path while $rG_{i,j}$ and $Out_{result_{i,j}}$ respectively refer to input data samples and output data samples. The dataset is represented as a matrix C with $J_M * r$ rows.

4) *Machine Learning Model*: The obtained edge relationship data rG is as samples, and divide them into training samples and prediction samples. The training samples are first normalized, then the training is carried out in the machine learning model after the model parameters are initialized, and the weighted average of the obtained is carried out. The parameters of the machine learning model were optimized by the predicted samples and weight, and the loss was calculated by error analysis based on the sample label value Out_{result} obtained by Algorithm 2.

5) *Correction*: The prediction value based on the machine learning model were analyzed by unique alarm code constraint. If the prediction is correct, no operation is performed, otherwise the class flip is performed, and the correct sequence is obtained.

In the Algorithm 1, the input data is reconstructed. In the Algorithm 2, the output data is reconstructed. In the Algorithm 3, based on this dataset jointly constructed by Algorithms 1 and 2, m-trails generated by machine learning algorithms are connected as mlm-trails.

A. Input Data Reconstruction

This study uses a unique data representation approach in which a single data instance is constructed as a composite matrix. This matrix structure is composed of three submatrices

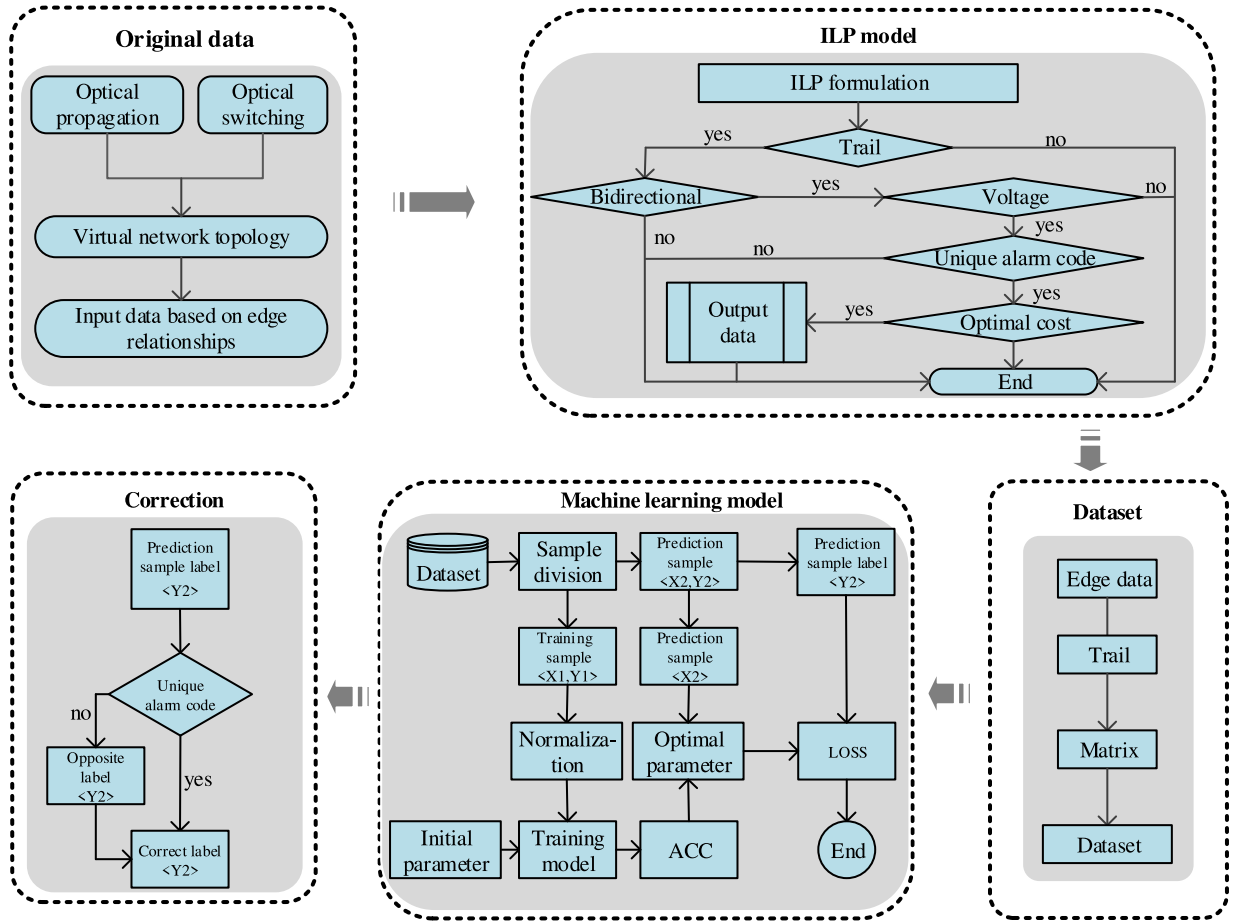


Fig. 1. Machine Learning-based Monitoring Trail Design Process.

of size $r * r$, which are spliced in the column direction to form a composite matrix $r * (3 * r)$. The row and column indexes of each submatrix are associated with a specific edge number, allowing the data structure to efficiently map the connection relationships in the network. The implementation process is shown in Algorithm 1:

Where, G is the adjacency matrix, which $G_{i,j}$ indicates whether there is an edge between nodes i and j . A relational matrix reG representing edges with the same starting point. rsG represents an edge relation matrix with the same end point. The end point of one side is the relationship matrix $resG$ of the starting point of the other side. A synthetic relation matrix rG , constructs the edge relation matrix of the new structure. In order to specifically illustrate the data modeling process, this paper takes six steps to explain its data construction.

Step 1: Initialize an empty matrix to store the converted data;

Step 2: Iterate through each element of the adjacency matrix G and perform the following operations:

- 1) If the value of the element is 1, it means that there is an edge, replace the element with the number of the current edge.
- 2) Record the row and column indexes of all non-zero elements, which will be used to populate the final composite matrix.

Step 3: Construct the first submatrix reG :

- 1) Traverse the non-zero elements of the row, and for each non-zero element, set its corresponding row index and column index position to 1.
- 2) Repeat this process until all non-zero elements of the row have been processed.

Step 4: Construct the second submatrix rsG :

- 1) Iterate over the non-zero elements of the first column, setting its corresponding row index and other fixed column index positions to 1 for each non-zero element.
- 2) Repeat this process until all the non-zero elements of the first column have been processed.

Step 5: Construct the third submatrix $resG$:

- 1) Traverse the remaining rows and columns, finding all non-zero elements.
- 2) For each non-zero element, set its corresponding row index and column index position to 1.
- 3) Repeat this process until all non-zero elements have been processed.

Step 6: Assemble the three submatrices by column direction to form a composite $r * (3 * r)$ matrix rG .

Therefore, for a network of 7 nodes and 13 edges, the resulting data set is a $13 * 39$ matrix, where each row represents an edge and each column represents a specific connection relationship.

Algorithm 1: Input Data Reconstruction.

Input : G .
Output: rG .

- 1 The number of rows in matrix G is denoted as r ;
Initialize the number of edges $numE=0$;
- 2 **for** ($i = 1; i \leq r; i++$) **do**
- 3 **for** ($j = 1; j \leq r; j++$) **do**
- 4 **if** $G[i][j] == 1$ **then**
- 5 $numE+=1$;
- 6 $G[i][j]=numE$;
- 7 **else**
- 8 Break 3;
- 9 **end**
- 10 **end**
- 11 **end**
- 12 Initializes three $numE*numE$ dimensional all-0 matrices:
 reG , rsG and $resG$;
- 13 **for** ($i = 1; i \leq r; i++$) **do**
- 14 **for** ($j = 1; j \leq r-1; j++$) **do**
- 15 **for** ($k = 1; k \leq r; k++$) **do**
- 16 **if** $G[i][j]! = 0 \ \&\& \ G[i][k]! = 0$ **then**
- 17 $reG[G[i][j], G[i][k]]=1$;
- 18 **else**
- 19 Break 16;
- 20 **end**
- 21 **if** $G[j][i]! = 0 \ \&\& \ G[k][i]! = 0$ **then**
- 22 $rsG[G[j][i], G[k][i]]=1$;
- 23 **else**
- 24 Break 15;
- 25 **end**
- 26 **end**
- 27 **end**
- 28 **end**
- 29 **for** ($i = 1; i \leq r; i++$) **do**
- 30 **for** ($j = 1; j \leq r; j++$) **do**
- 31 **if** $G[i][j]! = 0$ **then**
- 32 **for** ($k = 1; k \leq r; k++$) **do**
- 33 **if** $G[j][k]! = 0$ **then**
- 34 $resG[G[i][j], G[j][k]]=1$;
- 35 **else**
- 36 Break 32;
- 37 **end**
- 38 **end**
- 39 **else**
- 40 Break 30;
- 41 **end**
- 42 **end**
- 43 **end**
- 44 **Return** rG .

B. Output Data Reconstruction

As shown in the flow chart in Fig. 1, this study obtains the values of decision variables by solving ILP, and generates the corresponding adjacency matrix according to the values of these decision variables. Then the adjacency matrix is traversed and

Algorithm 2: Output Data Reconstruction.

Input : G .
Output: Out_{result} .

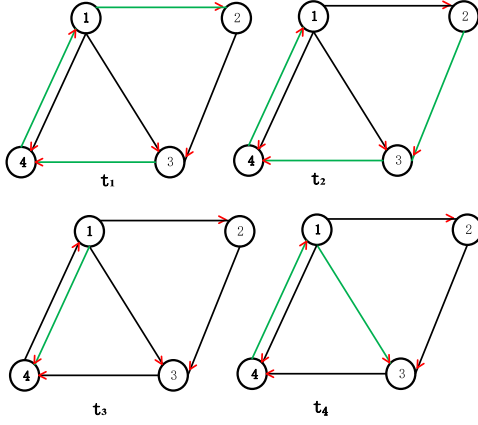
- 1 Initialize $Out_{result}[], k=0, str[]$;
- 2 The m-trails of ILP solution is obtained and represented
as e_{uv} and e_{vu} ;
- 3 $AP=e_{uv}+e_{vu}$;
- 4 **for** ($j; i \leq sizeof(AP[i])/sizeof(AP[i][0]); j++$) **do**
- 5 $AP[i][j] = str + k$;
- 6 $K += str + k$;
- 7 **end**
- 8 $Char * p = str$;
- 9 **while** ($*p! = 0$) **do**
- 10 **if** $*p ==$ **then**
- 11 $move(p, p + 1, strlen(p))$;
- 12 **else**
- 13 $p++$;
- 14 **end**
- 15 **end**
- 16 $num = atoll(str)$;
- 17 $Out_{result} = i$
- 18 **for** ($j = 0; j \leq sizeof(AP[i])/sizeof(AP[i][0]); j++$) **do**
- 19 $Out_{result}[i] + strlen(Out_{result}[i]) = AP[i][j]$;
- 20 $Out_{result}[i] + strlen = Out_{result}[i]$.
- 21 **end**

corresponding labels are generated according to the adjacency matrix. The specific implementation is shown in Algorithm 2.

This process starts by storing each vector in the trails scheme in the matrix AP (line 3), then traverses the matrix AP , converting the elements of each row into a string (lines 4-7). After removing the Spaces between the strings, concatenate them into a long string and get the decimal code representation of that long string (lines 8-16). Then store the AP line number, the value of the current AP line, and its decimal code in each line of Out_{result} . Through the above steps, the data set required for the training of the prediction model in this study can be obtained.

1) *Unique Alarm Code*: To clarify every link, we label every edge between node u and node v in the network $G(V, E)$ as L_{uv} . Then, t_j is set to represent the traversed situation in the network where j is the index of all monitoring trails, if there is a link traversed by the trail, t_j takes 1, otherwise 0. In this paper, we introduced a set of alarm code to realize clear localization. In order to ensure the uniqueness of alarm codes, bit comparison is needed. To address the issue of bit comparison, we have introduced a decimal alarm code. The corresponding decimal codes are shown in Fig. 2(c), where the alarm code field is a candidate set of decimal codes, including $\{1, 8, 4, 2, 3, 11\}$. For example, alarm code of $L_{4,1} = 2^3 + 2^1 + 2^0 = 11$. Consequently, it indicates that there is a failure in link $L_{4,1}$ when alarm code is 11.

If there are r links in the topology $G(V, E)$, then at least $(\lceil \log_2^r \rceil + 1)$ bits of alarm code is required for clear failure



(a) The mml-trail Solution for Virtual Hybrid Network with 4 Nodes and 6 Edges which is Constructed by All-optical DCNs.

Link	t_4	t_3	t_2	t_1	Decimal
(L ₁₂)	0	0	0	1	1
(L ₁₃)	1	0	0	0	8
(L ₁₄)	0	1	0	0	4
(L ₂₃)	0	0	1	1	3
(L ₃₄)	0	0	1	1	3
(L ₄₁)	1	0	1	1	11

(b) The Prediction Value of Monitoring Trails and Decimal Alarm Code based on the Proposed CNN-BiLSTM-SE Attention Model

Link	t_4	t_3	t_2	t_1	Decimal
(L ₁₂)	0	0	0	1	1
(L ₁₃)	1	0	0	0	8
(L ₁₄)	0	1	0	0	4
(L ₂₃)	0	0	1	0	2
(L ₃₄)	0	0	1	1	3
(L ₄₁)	1	0	1	1	11

(c) The Corrected Value of Monitoring Trails and Decimal Alarm Code based on the Proposed mml-trail Algorithm

Fig. 2. Illustrative Example.

localization. So, the lower limit of number of trails is:

$$J_m = (\lfloor \log_2^r \rfloor + 1) \quad (2)$$

However, there will not be a solution if the number of trails is smaller than the value required for the optimal solution. Hence, we need an integer J_M as the upper limit of the number of trails, which should be large enough to achieve optimality.

2) *Notation List*: For a given optical network $G(V, E)$, we describe the process of localizing failures as an ILP problem. The constraints notation list is as follows:

J_M : The maximum number of m-trails allowed in the solution.

j : M-trail index, where $j \in \{0, 1, \dots, J_M - 1\}$.

E : The set of all links in topology.

E_U : The set of all links in the upper triangulation in the topology.

E_D : The set of all links in the lower triangulation in the topology.

V : The set of all nodes in topology.

V_U : The set of all nodes in upper triangulation in topology.

V_D : The set of all nodes in lower triangulation in topology.

c_{uv} : Predefined cost of a supervisory wavelength on a single link (u, v) .

L : Predefined length limit of each m-trail.

γ : Predefined cost ratio of a monitor to a supervisory wavelength link.

d : A predefined small positive value ($0 \leq d \leq r^{-1}$). The minimum increase in the voltage value in a voltage constraint.

B : Lower bound of the monitoring cost.

ρ : A predefined small constant and $0 < \rho < 2^{-J_M}$.

e_{uv}^j : Binary variable. If $u \rightarrow v$ is an on-trail vector of m-trail t_j , then 1 is required, otherwise 0 is required.

m^j : Binary variable. If t_j is an m-trail, then 1 is required, otherwise 0 is required.

t_u^j : Binary variable. If node u is the source node on m-trail t_j , then 1 is required, otherwise 0 is required.

r_u^j : Binary variable. If node u is the end node on m-trail t_j , then 1 is required, otherwise 0 is required.

z_u^j : Binary variable. If node u is traversed by m-trail t_j , then 1 is required, otherwise 0 is required.

q_{uv}^j : Nonnegative fractional variable. It is the voltage of $u \rightarrow v$ vector on m-trail t_j . If is not an on-trail vector of t_j , then 0 is required.

a_{uv} : General integer variable. It is the decimal alarm code assigned to link (u, v) .

f_{uv}^{xy} : Binary variable. For two distinct links (u, v) and (x, y) , if $a_{uv} > a_{xy}$, then 1 is required, and if $a_{uv} < a_{xy}$, then 0 is required.

3) *ILP Formulation*:

$$\min \left\{ \gamma \times \sum_j m^j + \sum_j \sum_{(c_{uv} \in E)} c_{uv} (e_{uv}^j + e_{vu}^j) \right\} \quad (3)$$

Subject to:

$$\sum_{u \in V} t_u^j \leq 1 \quad (4)$$

$$\sum_{u \in V} r_u^j = 1 \quad (5)$$

$$\sum_{(u,v) \in E} (e_{uv}^j + e_{vu}^j) - \sum_{(u,w) \in E} (e_{uw}^j + e_{wu}^j) = t_u^j - r_u^j, \forall u \in V, \forall j \quad (6)$$

$$e_{uv}^j + e_{vu}^j \leq 1, \forall (u, v) \in E_U, \forall j \quad (7)$$

$$e_{uv}^j \leq z_u^j, \forall u \in V : (u, v) \in E_U, \forall j \quad (8)$$

$$e_{uv}^j + e_{vu}^j \leq z_u^j, \forall u \in V : (u, v) \in E_U \cap (v, u) \in E_U, \forall j \quad (9)$$

$$e_{uv}^j \leq z_u^j, \forall u \in V : (u, v) \in E_D, \forall j \quad (10)$$

$$e_{uv}^j + e_{vu}^j \leq z_u^j, \forall u \in V : (u, v) \in E_D \cap (v, u) \in E_D, \forall j \quad (11)$$

$$0 \leq q_{uv}^j \leq e_{uv}^j, \forall (u, v) \in E_U, \forall j \quad (12)$$

$$0 \leq q_{vu}^j \leq e_{vu}^j, \forall (u, v) \in E_D, \forall j \quad (13)$$

$$r_u^j + \sum_{(u,v) \in E} (q_{uv}^j + q_{vu}^j) - \sum_{(u,w) \in E} (q_{uw}^j + q_{wu}^j) \geq d \times z_u^j \quad (14)$$

$$\sum_j^{m^j} \geq \lfloor \log_2^r \rfloor + 1 \quad (15)$$

$$c_{uv}(e_{uv}^j + e_{vu}^j) \leq L, \forall j \quad (16)$$

$$\sum_j \sum_{(u,v) \in E} c_{uv}(e_{uv}^j + e_{vu}^j) \geq 2r - \sum_j^{m^j} \quad (17)$$

$$\gamma \sum_j^{m^j} + \sum_j \sum_{(u,v) \in E} c_{uv}(e_{uv}^j + e_{vu}^j) \geq B \quad (18)$$

$$r_u^j \leq \sum_{u \in V}^{t_u^j} \leq 1, \forall j \quad (19)$$

$$\sum_j \sum_{(u,v) \in E} c_{uv}(e_{uv}^j + e_{vu}^j) \leq r \quad (20)$$

$$a_{uv} = \sum_j^{2^j} (e_{uv}^j + e_{vu}^j), \forall (u, v) \in E_U \quad (21)$$

$$a_{uv} = \sum_j^{2^j} (e_{uv}^j + e_{vu}^j), \forall (u, v) \in E_D \quad (22)$$

$$a_{uv} \geq 1, \forall (u, v) \in E_U \quad (23)$$

$$a_{uv} \geq 1, \forall (u, v) \in E_D \quad (24)$$

$$\rho + \rho(a_{uv} - a_{xy}) \leq f_{uv}^{xy}, \forall (u, v), (x, y) \in E_U : (u, v) \neq (x, y) \quad (25)$$

$$\rho + \rho(a_{xy} - a_{uv}) \leq 1 - f_{uv}^{xy}, \forall (u, v), (x, y) \in E_U : (u, v) \neq (x, y) \quad (26)$$

$$\rho + \rho(a_{uv} - a_{xy}) \leq f_{uv}^{xy}, \forall (u, v), (x, y) \in E_D : (u, v) \neq (x, y) \quad (27)$$

$$\rho + \rho(a_{xy} - a_{uv}) \leq 1 - f_{uv}^{xy}, \forall (u, v), (x, y) \in E_D : (u, v) \neq (x, y) \quad (28)$$

$$\rho + \rho(a_{uv} - a_{xy}) \leq f_{uv}^{xy}, \forall x \in V_U, y \in V_D \quad (29)$$

$$\rho + \rho(a_{xy} - a_{uv}) \leq 1 - f_{uv}^{xy}, \forall x \in V_U, y \in V_D \quad (30)$$

Objective function in (3) aims to minimizing the whole cost. The first term is the cost of all monitors, and the second term represent the sum of predefined cost of the supervisory wavelength. Constraints (4) and (5) stipulate a source-end node pair in each m-trail t_j . Constraint (6) is a balancing constraint, which ensures that the traffic of each node is propagated correctly between different triangulation. Constraint (7) formulates at most a single directed vector on each link (u, v) , either $u \rightarrow v$ or $v \rightarrow u$. Constraints (8)–(11) formulate that a node is traversed by an m-trail t_j properly in different environments (i.e., Both $u \rightarrow v$ and $v \rightarrow u$ belong to the upper triangulation or belong to different triangulation). Constraints (12)–(13) ensure that only an on-trail vector can have a positive voltage value, both in the upper triangulation and in the lower. Constraint (14) formulates that the total flow must meet the minimum required quality of service requirements. Constraint (15) specifies the lower bound on the required number of m-trails. Constraint (16) means that if there is $v \rightarrow u$ or $v \rightarrow u$ on the m-trail, then the total cost cannot exceed the link length limit. Constraint (17) says that if the solution contains $\sum_j^{m^j}$ m-trails, only $\sum_j^{m^j}$ links can have a cover times of 1, and all other links must have a cover times no smaller than 2. Constraint (18) restrains that the total bandwidth in the system must be larger than or equal to the lower bound on the monitoring cost. Constraint (19) indicates that, for each node u , out of all service categories j , the number of times node u is selected as the end point of the service cannot exceed the number of times node u is used as the starting point of the service. Constraint (20) indicates that, for each node u , the total amount of traffic between node u and other nodes in the service category j cannot exceed r . Constraints (21)–(22) convert binary code to the decimal, and constraints (23)–(24) prevent duplicate alarm codes. Eventually, constraints (25)–(30) ensure a unique alarm code for each link failure.

C. Machine Learning Algorithm

We use the dataset generated above and train the learning model using the following machine learning algorithms:

Back Propagation (BP) [25] is a multi-layer feedforward neural network, whose basic principle includes two processes: forward propagation and backward propagation. The learning process gradually approaches the goal by continuously calculating the changes in network weights and biases in the direction of decreasing slope relative to the error function.

Random Forest (RF) [26] is a classifier algorithm that constructs multiple decision trees during the training phase and outputs the model of a single tree as the class label. It is very suitable for multi class classification problems, such as link fault identification and localization.

Convolutional Neural Networks (CNN) [27] is to gradually extract high-level features from images through convolution and pooling operations. During the training process, CNN constantly adjusts the parameters of the convolution kernel and the fully connected layer through the backpropagation algorithm to minimize the error between the predicted value and the actual value.

Long Short-Term Memory (LSTM) [28] is a type of recurrent neural network suitable for processing sequence data. At the

Algorithm 3: Mlm-Trail Algorithm.

Input : Dataset C .
Output:
 mlm-trails.

Step 1: CBSA algorithm:
 Prediction value of m-trails is obtained based on the proposed CBSA algorithm.

Step 2: Correction:
 The set DV includes the decimal values corresponding to the optical path r obtained through the CBSA algorithm; The set ϖ includes all decimal values that can be represented when the number of m-trails is J_M , such as $\varpi = \{2^0, 2^1, \dots, 2^{J_M}\}$.

```

1 if length(unique(DV)) < numel(DV) then
2   for (i = 1; i ≤ r; i++) do
3     for (j = i + 1; j ≤ r - 1; j++) do
4       if DVi = DVj then
5         DVi is modified as DVi* by selecting
           value from set  $\xi = \varpi - DV$ ,
            $\{\xi/\xi \in \varpi, \xi \notin DV\}$ , so that there are no
           duplicate values in set  $DV$ , and the
           decimal code satisfies the principle of
           uniqueness.
6       end
7        $\varpi = \varpi - DV_i^*$ ;
8       mlm-trails are generated by modifying
           m-trails of the corresponding updated
           decimal values.
9     end
10  end
11  for (i = 1; i ≤ JM; i++) do
12    Traverse JM mlm-trail to meet the voltage
        constraint.
13  end
14  for (i = 1; i ≤ r; i++) do
15    Traverse r optical path to assign at least one
        mlm-trails.
16  end
17 end
18 else if incomplete mlm-trails then
19   mlm-trails are modified by meeting voltage
        constraints.
20   Break 1;
21 end
22 else if optical path without assigned mlm-trail then
23   All optical paths in the virtual network topology are
        corrected and assigned at least one mlm-trail.
24   Break 1;
25 end
26 Return mlm-trails.
```

heart of the LSTM algorithm is a loop unit that learns long-term dependencies. The core component of the LSTM is a cell state, which controls the flow of information in and out through a structure called a gate.

Extreme Learning Machine (ELM) [29] randomly selects input layer weights and hidden layer biases, and outputs layer

weights to minimize the loss function composed of the training error term and the regularization term of the output layer weight norm. Based on Moore Penrose (MP) generalized inverse matrix theory, analytical calculations were performed on it.

Support Vector Machine (SVM) [30] is a supervised machine learning technique that attempts to classify data points into two different classes by identifying the optimal separating hyperplane. By constructing multiple hyperplanes, it can be extended to multiple types of problems.

Graph Convolutional Network (GCN) [17] performs convolution operations on graph structured data to capture complex relationships and features between nodes in the graph. It updates the representation of each node through the information of neighboring nodes, which can be seen as aggregating and updating information in the graph.

Latent Dirichlet Allocation (LDA) [27] is a probabilistic model used to analyze hidden topics in text data. It assumes that each document is composed of multiple topics mixed together, and each topic is composed of multiple words. LDA can infer the probability relationship between topics and words by analyzing the distribution of word occurrences in documents.

D. Mlm-Trail Algorithm

We have noticed that although the above machine learning models can quickly identify detection trace solutions for classification problems, they cannot achieve 100% accurate prediction, which has always been a major challenge in using machine learning methods to solve fault localization problems.

In this article, we propose an mlm-trail algorithm that can quickly obtain results while achieving accurate failure localization of the optical path in all-optical DCNs. Our proposed mlm-trail algorithm consists of two steps:

Firstly, a hybrid neural network model CNN-BiLSTM-SE Attention (CBSA) is proposed, which combines convolutional neural network (CNN), bidirectional long short-term memory network (BiLSTM), and SE (Squeeze-and-Excitation) attention mechanism to quickly predict the monitoring trail results. This model combines the local feature extraction capability of CNN, the sequence feature extraction capability of BiLSTM, and the global spatial feature extraction capability of SE (Squeeze and Excitation) attention mechanism, aiming to effectively extract multi-level features from data and improve classification accuracy. Combining the structures of these three layers, the model first captures the spatial features of the input monitoring trail sequence through a CNN layer, then captures the temporal correlation of the monitoring trail sequence through a bidirectional BiLSTM layer, and finally uses a SE Attention layer to better understand the correlations within the sequence. This comprehensive structure can more comprehensively process the monitoring trail sequence data.

Secondly, based on the predicted value of the monitoring trail obtained by the above CBSA machine learning algorithm, it is corrected. Algorithm 3 shows the pseudo code of the second step of proposed mlm-trail algorithm. Firstly, the set DV includes these decimal values corresponding to each monitoring trail which is calculated based on the monitoring trail generated by

machine learning algorithms. Then, based on the obtained set DV , the monitoring trail values are corrected according to three principles. Firstly, the principle of decimal code uniqueness must be met. Once duplicate decimal codes are found, the values in the set DV are modified to be unique. Then, it is necessary to meet the principle of integrity of the monitoring trail. Once an incomplete monitoring trail is found, the monitoring trail is modified by meeting voltage constraints to achieve completeness. Finally, it is necessary to meet the principle that each optical path is covered by the monitoring trail. Once it is found that there is an optical path that is not covered by the monitoring trail, optical path in the virtual network topology is corrected and assigned at least one monitoring trail.

Now we analyze the time complexity of step 2 of mlm-trail algorithm. Firstly, we analyze the principle of decimal code uniqueness, which sets two “for” loop to ensure that there are no duplicate values in set DV , and the time complexity is $O(r^2)$. Secondly, we analyze the principle of integrity of the monitoring trail, which sets one “for” loop to ensure that all mlm-trails meet the voltage constraints, and the time complexity is $O(J_M)$. Finally, we analyze the principle that each optical path is covered by the mlm-trail, which sets one “for” loop to ensure that each optical path is assigned by at least one mlm-trails, and the time complexity is $O(r)$. The number of m-trails J_M is less than or equal to the number of optical path r , so the time complexity of the mlm-trail algorithm is $O(r^2)$. The time complexity of step 1 of mlm-trail algorithm is the machine learning, which is mainly directly proportional to the size of the dataset matrix C . Although the time complexity of machine learning algorithms is linear, the size of the input dataset (the row size of the input dataset is $J_M * r$, and the column size is $r * (3 * r)$) is much larger than the number of edges r . Therefore, the final time complexity of the mlm-trail algorithm depends on proposed machine learning part being $O(3J_M * r^3)$.

E. Illustrative Example

Fig. 2(a) gives a mlm-trails solution for hybrid network which includes 4 nodes and 6 edges. Fig. 2(b) shows binary alarm coding and decimal alarm codes for Fig. 2 based on proposed CNN-BiLSTM-SE Attention model, where 1 represents the mlm-trail passing through this link and 0 represents mlm-trail not passing through this link. We can see that four mlm-trails t_1, t_2, t_3, t_4 (each with a dedicated monitor) are required to distinguish all the link failures. Concretely, $t_1 : 3 \rightarrow 4 \rightarrow 1 \rightarrow 2$; $t_2 : 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$; $t_3 : 1 \rightarrow 4$; $t_4 : 4 \rightarrow 1 \rightarrow 3$. Although the failure localization time is small, only 0.1us, the decimal alarm codes of links (2, 3) and (3, 4) are duplicated, which does not comply with the principle of uniqueness of decimal alarm codes, so it cannot be accurately located, with an failure localization accuracy of 83.3%. To achieve the failure localization of each optical path, the solution of the proposed CNN-BiLSTM-SE Attention model needs five monitors, three for links (1, 2), (1, 3), (1, 4) and (4, 1), and two additional link-based monitor for the link (2, 3) and (3, 4) which have the same decimal alarm codes.

Fig. 2(c) shows binary alarm coding and decimal alarm codes for Fig. 2 by algorithm 3 based on the prediction value of

CNN-BiLSTM-SE Attention model. According to the principle of uniqueness of decimal alarm codes, the t_1 on link (2, 3) is removed, that is, the t_1 value corresponding to link (2, 3) is changed from 1 to 0, so that the decimal alarm code value corresponding to link (2, 3) is changed from 3 to 2. This is different from the decimal alarm code corresponding to link (3, 4) to achieve 100% failure localization accuracy. Although algorithm 3 is added and the failure localization time is increased compared to single machine learning algorithms, the failure localization cost is also reduced. The solution of the proposed mlm-trail only needs four monitors for links (1, 2), (1, 3), (2, 3), (3, 4), (1, 4) and (4, 1) to achieve 100% failure localization accuracy.

IV. PERFORMANCE STUDY

A. Simulation Setting and Data Collection

This experiment was implemented using the Yalmip toolbox, and the ILP model was optimized using ILOGCPLEXV12.7.0. The hardware environment is Intel(R) Core(TM) i7-9700 CPU @3.00 GHz with 16.0 GB memory. Through a detailed comparison on 10,000 virtual network topology G (the number of nodes is 7, 40, and 100, and the number of edges is randomly generated) as the original data, this study investigates the performance of BP model, RF model, CNN model, LSTM model, ELM model, SVM model, GCN model, LDA model, CNN-LSTM model and CNN-BiLSTM-SE Attention model adopted in this paper in terms of four key performance indicators: accuracy, precision, recall and F1 score. Table I provides a detailed comparison of the different models on various metrics. We conducted 100 simulations and compared and analyzed the experiments based on failure localization accuracy, failure localization time and failure localization cost.

B. Performance Metrics Under Different Machine Learning Models

In order to comprehensively evaluate the performance of the model, we use the following four quantitative evaluation metrics: Accuracy (ACC), Precision (PRE), Recall (REC), and F1 Score. These metrics are based on the concept of a Confusion Matrix, which is a 2×2 matrix that describes how well a model predicts something. There are four elements in the confusion matrix: True Positive (TP), False Positive (FP), True Negative (TN), and False Negative (FN). The above four evaluation indicators are calculated using the following formula:

Accuracy: Accuracy is the proportion of the total number of samples for which all predictions are correct (both positive and negative). The formula is as follows:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (31)$$

Where TP is the positive class and the prediction is the number of positive classes; FN is the positive class predicted as the number of negative classes; FP is the negative class predicted as the number of positive classes; TN is the negative class predicted as the negative class number.

TABLE I
PERFORMANCE COMPARISON UNDER DIFFERENT MODEL

Model	ACC	PRE	REC	F ₁ -Score
BP	78%	75.38%	89.09%	81.67%
RF	74%	72.88%	81.13%	76.79%
CNN	74%	75%	77.78%	76.36%
LSTM	78%	80%	73.47%	76.6%
ELM	73%	71.67%	81.13%	76.11%
SVM	77%	76.47%	88.14%	81.89%
GCN	73%	72.22%	76.47%	74.29%
LDA	73%	79.25%	79.25%	79.25%
CNN-LSTM (C-LSTM)	76%	77.23%	79.25%	79.25%
CNN-BiLSTM-SE Attention (CBSA)	83%	84.26%	88.9%	88.9%

Precision: Precision is the fraction of examples predicted as positive that actually belong to the positive class. The formula is as follows:

$$PRE = \frac{TP}{TP + FP} \quad (32)$$

Recall: Recall is the proportion of samples that are actually positive that are correctly predicted to be positive. The formula is as follows:

$$REC = \frac{TP}{TP + FN} \quad (33)$$

F₁-Score: The F₁-Score is the harmonic mean of precision and recall. The formula is:

$$F1 = \frac{2 * PRE * REC}{PRE + REC} \quad (34)$$

In this section, we evaluate the performance of different machine learning models with the 7-node DCNs (which reconstructed 951 sample data consisting of topological edge relationships and m-trails). The size of the generated dataset is adjusted to 951x40, and 851 samples are used for training, while 100 are used for testing. We quantified the accuracy, precision, recall, and F1-Score metrics and presented the experimental data in Fig. 3.

The accuracy of the BP model (which is trained for 10000 epochs and a learning rate of 0.01) is about 78%, the precision is about 75.38%, the recall rate is about 89.09%, and the F1-Score is about 81.67%.

The accuracy of the RF model (which is trained for the number of trees is 50, and the minimum number of leaves is 1) is about 74%, the precision is about 72.88%, the recall rate is about 81.13%, and the F1-Score is about 76.79%.

The accuracy of the CNN model (which is trained with Adam optimization for 500 epochs and a initial learning rate of 0.001, then the learning rate decreased to 0.0001 after 450 epochs) is about 74%, the precision is about 75%, the recall rate is about 77.78%, and the F1-Score is about 76.36%.

The accuracy of the LSTM model (which is trained with Adam optimization for 1000 epochs and a initial learning rate of 0.1, then the learning rate decreased to 0.01 after 750 epochs) is about 78%, the precision is about 80%, the recall rate is about 73.47%, and the F1-Score is about 76.6%.

The accuracy of the ELM model (which is trained for sigmoid activation function, and the number of hidden layer nodes is 50)

is about 73%, the precision is about 71.67%, the recall rate is about 81.13%, and the F1-Score is about 76.11%.

The accuracy of the SVM model (which is trained with a penalty parameter of 10, and radial basis function parameter is 0.01) is about 73%, the precision is about 76.47%, the recall rate is about 88.14%, and the F1-Score is about 81.89%.

The accuracy of the GCN model (which is trained with Adam optimization for 100 epochs and a learning rate of 0.01) is about 73%, the precision is about 72.22%, the recall rate is about 76.47%, and the F1-Score is about 74.29%.

The accuracy of the LDA model (The dataset is randomly shuffled for training, with the training set accounting for 70% of the dataset) is about 73%, the precision is about 79.25%, the recall rate is about 79.25%, and the F1-Score is about 79.25%.

Among the machine learning models, BP, LSTM and SVM models outperform RF, ELM, GCN and LDA models with a accuracy of about 78%, a precision of about 80%, a recall of about 89.09% and F1-Score of about 81.89% shown in Fig. 3.

C. The Performance Analysis of Proposed CNN-BiLSTM-SE Attention Model Under Different Learning Rate

In this section, we evaluate the performance of proposed CNN-BiLSTM-SE Attention (CBSA) model under different learning rate with the 7-node DCNs. For 951 sample data with 5-fold cross-validation, 80% of the samples are used for training, while 20% are used for testing.

We quantified the accuracy and loss metrics and presented the experimental data in Fig. 4. The experiment runs for 100 epochs using Adam optimization, with 6 iterations per epoch, reaching a total of 6000 iterations. When the learning rate is 0.1 and 0.01, as the number of iterations increases, the accuracy and loss of the three models remain relatively smooth, and the accuracy and loss of proposed CNN-BiLSTM-SE Attention model in this paper are better than CNN-LSTM and LSTM models. When the learning rate is 0.001 and 0.0001, as the number of iterations increases, the accuracy and loss of the three models gradually become smoother, and the accuracy and loss values of the proposed CNN-BiLSTM-SE Attention model in this paper are better than CNN-LSTM and LSTM models.

It can be concluded that as the learning rate decreases, the more iterations the model has, the smoother the accuracy and loss. But when the learning rate is 0.0001 and the number of

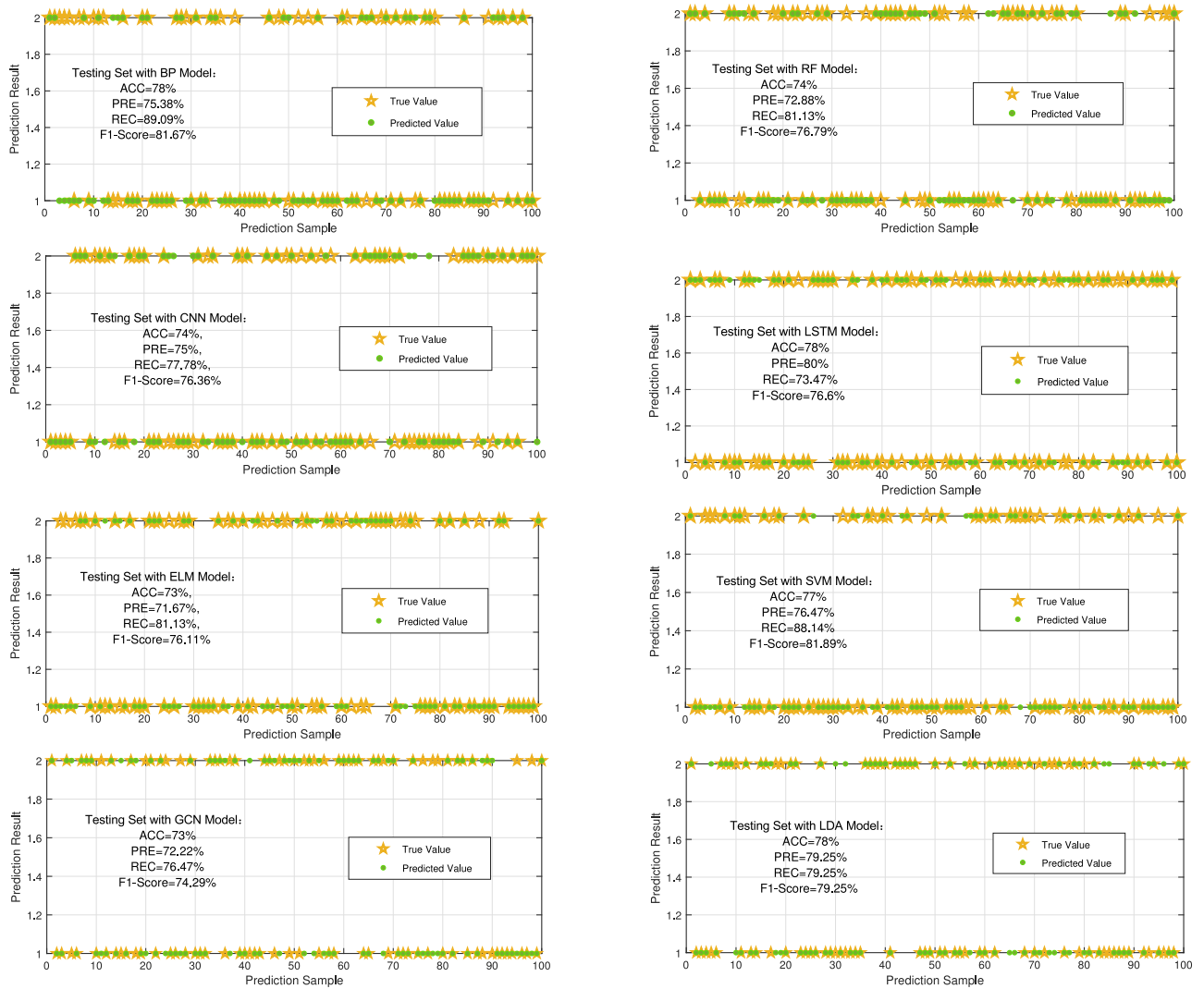


Fig. 3. Performance Metrics of Testing Set under Different Machine Learning Models{BP, RF, CNN, LSTM, ELM, SVM, GCN, LDA}.

iterations is less than 1000, the accuracy and loss values no longer follow the original linear rules.

Fig. 5 shows that the performance metrics of testing set under different machine learning models. Among them, the performance of proposed CNN-BiLSTM-SE Attention and CNN-LSTM models are superior to other machine learning models.

D. The Performance Analysis of the Proposed Mlm-Trail Algorithm With Exist Algorithms

We define the failure localization accuracy of all algorithms as the ratio of the effective number of monitoring trails to the total number. In Fig. 6, we present the failure localization accuracy of all algorithms for the three type networks. For the 10000 virtual network topologies of 7-node, 40-node, and 100-node all-optical DCN, the failure accuracy cost of LSTM algorithm is 80, while the three values of other traditional machine learning algorithms are not consistent, ranging from 73 to 78. This is because the dataset in this article consists of edge relationships

and monitoring trails sequences, and the LSTM algorithm is suitable for processing sequence data. It can be seen that the m-trail and mlm-trail algorithms perform better than the machine learning model with failure localization accuracy of 100% for the 7-node network, 40-node network and the 100-node virtual network topology structure by all-optical DCNs, respectively. This high failure localization accuracy results in the superior performance in identifying and localizing the optical path failure of all-optical DCNs.

The time taken to localize an optical path failure in the network upon its occurrence is an important metric as this affects the failure recovery time. The faster the optical path failure localization, the less the impact of the fault on the all-optical DCNs. In Fig. 7, we present the failure localization time of all algorithms for the three type networks. It can be seen that the m-trail algorithm perform worse than the other algorithms with failure localization time of 10000us, 18000us and 7000us for the 7-node network, 40-node network and the 100-node virtual network topology structure by all-optical DCNs, respectively. The optical

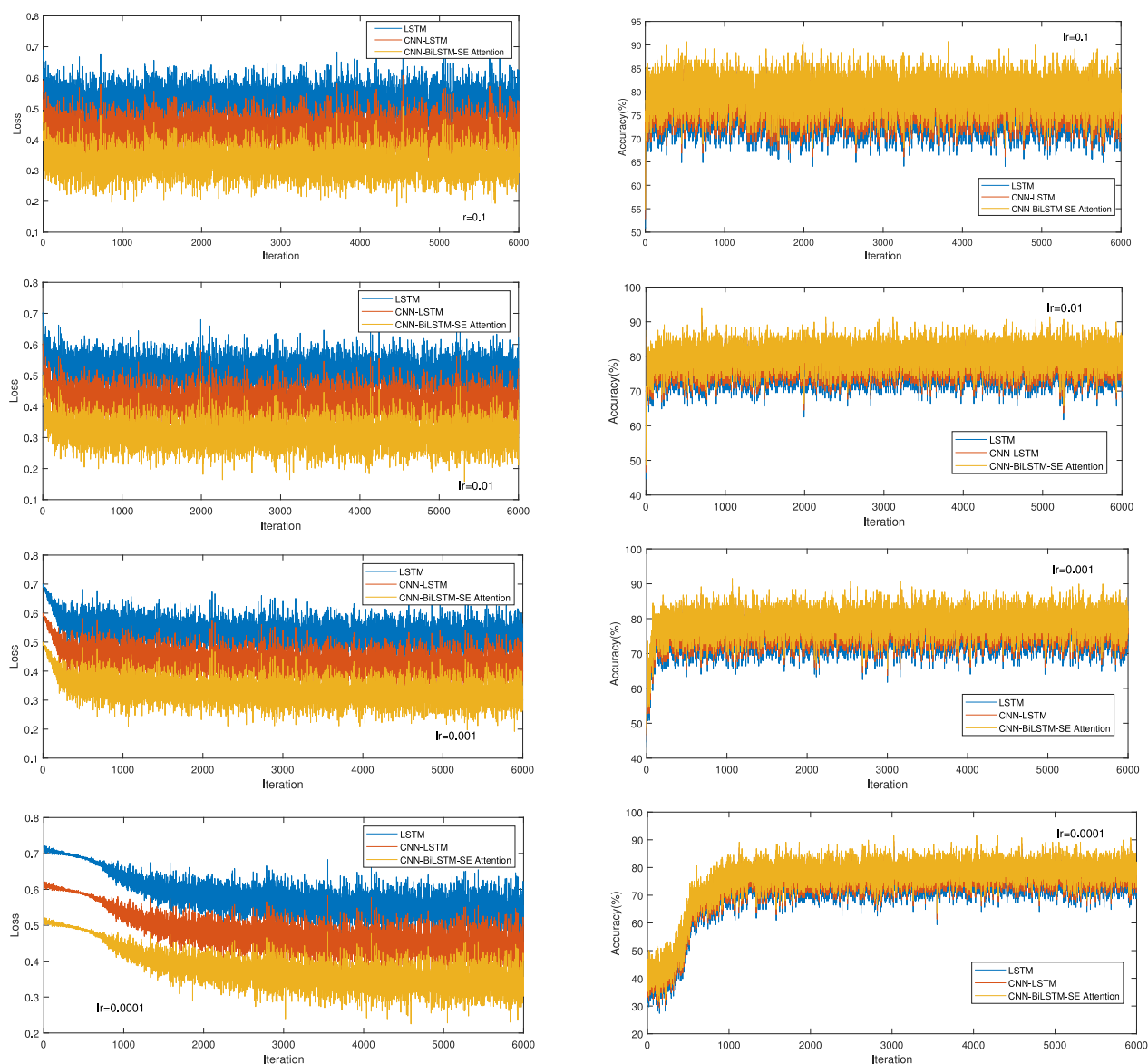


Fig. 4. Loss and Accuracy of Proposed CNN-BiLSTM-SE Attention Model under Different Learning Rate{0.1, 0.01, 0.001, 0.0001}.

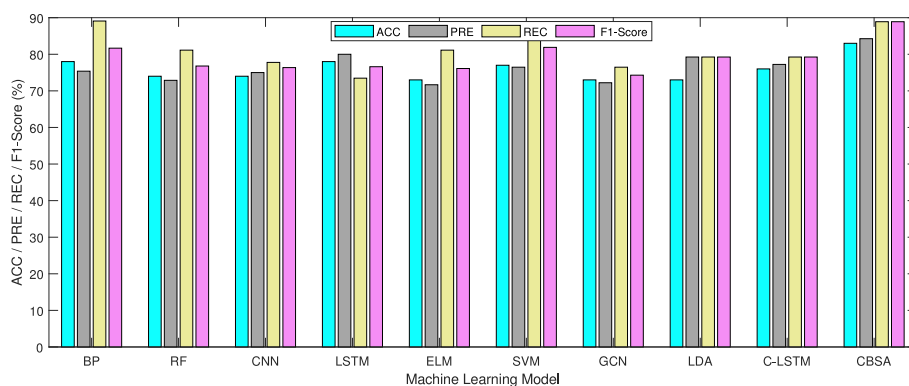


Fig. 5. Performance Comparison under Different Machine Learning Models.

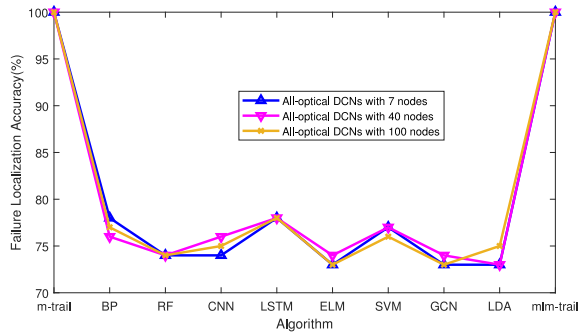


Fig. 6. Failure Localization Accuracy of All-optical DCNs under Different Nodes{7, 40, 100}.

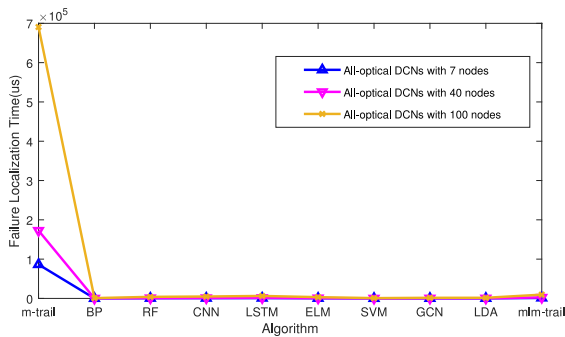


Fig. 7. Failure Localization Time of All-optical DCNs under Different Nodes{7, 40, 100}.

path failure localization time of the mml-trail algorithm is not significantly different from other machine learning algorithms. Because the time complexity of the mml-trail algorithm depends on the part of machine learning algorithm, the time complexity of the mml-trail algorithm is not significantly different from that of traditional machine learning algorithms. But the m-trail algorithm is implemented through ILP model, which belongs to NP-hard problems, and its computational complexity is much higher than machine learning algorithms.

The proposed method does have an impact on the bisectional bandwidth of DCN, and the network operator cannot just maximize the throughput anymore because of the failure localization method. The failure localization cost consists of the monitor cost and the bandwidth cost (which is denoted by the cover length of wavelength for monitoring optical path). In Fig. 8, we present the failure localization cost of all algorithms for the three type networks. It can be seen that the m-trail and mml-trail algorithms perform better than the other algorithms with failure localization cost of about 50, 150 and 300 for the 7-node network, 40-node network and the 100-node virtual network topology structure by all-optical DCNs, respectively. We can see that the cost difference between traditional machine learning models is very small. Although their failure localization time is relatively short, their common disadvantage is low failure localization accuracy, which requires the additional placement of a large number of alarms. The cost of alarm is based on the number of optical paths in the virtual network topology, which is a constant value,

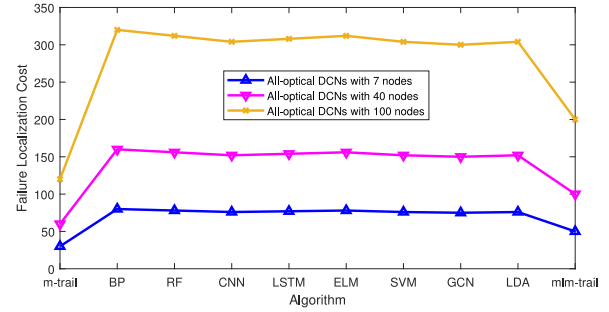


Fig. 8. Failure Localization Cost of All-optical DCNs under Different Nodes{7, 40, 100}.

so the cost difference is very small. The mml-trail algorithm has high failure localization accuracy while ensuring low failure localization time, and the accuracy of fault localization can be improved by adding pre-defined monitoring wavelengths instead of expensive alarms.

V. CONCLUSION

We proposed a mml-trails algorithm to achieve fast and accuracy optical path failure localization in all-optical DCNs. mml-trail firstly train the learning model based on the constructed dataset, which relies on topological edge relationships and the optimal solution generated by the m-trail algorithm using machine learning models (BP, RF, CNN, LSTM, ELM, SVM, GCN, LDA). Hybrid machine learning models (CNN-LSTM and CNN-BiLSTM-SE Attention) were proposed to achieve fast generation of monitoring trail. Among them, the dataset constructed in this paper can significantly improve the ability of the selected model in utilizing network topology information, and effectively improve the accuracy of fault prediction. Based on the constraints of full coverage of monitoring trail and uniqueness of alarms, the generated output results based on machine learning are modified to achieve unambiguous localization for each optical path. Numerical results showed that the proposed mml-trail scheme outperforms m-trail in localization speed and scalability, and also outperforms machine learning algorithm in accuracy and cost.

REFERENCES

- [1] P. Cao et al., "Threshold-based routing-topology co-design for optical data center," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2870–2885, Dec. 2023.
- [2] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [3] Stanford HAI, "Artificial Intelligence Index Report 2023," Stanford Univ., Stanford, CA, USA, 2023.
- [4] S.M. Srinivasan, T. Truong-Huu, and M. Gurusamy, "Machine learning-based link fault identification and localization in complex networks," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6556–6566, Aug. 2019.
- [5] H. Zeng, C. Huang, and A. Vukovic, "A novel fault detection and localization scheme for mesh all-optical networks based on monitoring-cycles," *Photon. Netw. Commun.*, vol. 11, no. 3, pp. 277–286, 2006.
- [6] E. A. Doumith, S. Al Zahr, and M. Gagnaire, "Monitoring-tree: An innovative technique for failure localization in WDM translucent networks," in *Proc. IEEE Glob. Telecommun. Conf.*, 2010, pp. 1–6.
- [7] B. Wu, K. L. Yeung, B. Hu, and P. H. Ho, "M2-CYCLE: An optical layer algorithm for fast link failure detection in all-optical mesh networks," *Comput. Netw.*, vol. 55, no. 3, pp. 748–758, 2011.

- [8] B. Wu, P. H. Ho, and K. L. Yeung, "Monitoring trail: On fast link failure localization in all-optical WDM mesh networks," *J. Lightw. Technol.*, vol. 27, no. 18, pp. 4175–4185, Sep. 2009.
- [9] S. Song et al., "Demonstration of intelligent hybrid FSO/RF system based on enhanced GRU prediction and real-world meteorological dataset," *J. Lightw. Technol.*, vol. 40, no. 21, pp. 7048–7059, Nov. 2022.
- [10] A. R. Mohammed, S. A. Mohammed, D. Côté, and S. Shirmohammadi, "Machine learning-based network status detection and fault localization," *IEEE Trans. Instrum. Meas.*, vol. 70, 2021, Art. no. 3521710.
- [11] H. Zeng, C. Huang, and A. Vukovic, "Monitoring cycles for fault detection in meshed all-optical networks," in *Proc. Workshops Mobile Wireless Netw./High Perform. Sci., Eng. Comput./Netw. Des. Architecture/Opt. Netw. Control Manage./Ad Hoc Sensor Netw.*, 2004, pp. 434–439.
- [12] Z. Cheng, X. Zhang, S. Shen, S. Yu, J. Ren, and R. Lin, "T-trail: Link failure monitoring in software-defined optical networks," *J. Opt. Commun. Netw.*, vol. 10, no. 4, pp. 344–352, 2018.
- [13] E. A. Doumith and S. Al Zahr, "Double-link failure localization based on the monitoring trail concept in WDM networks," *Proc. 2012 Symp. Broadband Netw. Fast Internet*, 2016, pp. 21–28.
- [14] Y. Xuan, Y. Shen, N. P. Nguyen, and M. T. Thai, "Efficient multi-link failure localization schemes in all-optical networks," *IEEE Trans. Commun.*, vol. 61, no. 3, pp. 1144–1151, Mar. 2013.
- [15] C. Ma, "Research on multi-link fault survivability technology in elastic optical network," Beijing Univ. Posts Telecommun., Beijing, China, 2016.
- [16] X. L. Wang, "Research on fault detection and localization algorithm in all-optical network," Xidian Univ., Xi'an, China, 2017.
- [17] Z. Wang et al., "Failure prediction using machine learning and time series in optical network," *Opt. Exp.*, vol. 25, no. 16, pp. 18553–18565, 2017.
- [18] Y. Zhao, B. Yan, D. Liu, Y. He, D. Wang, and J. Zhang, "SOON: Self-optimizing optical networks with machine learning," *Opt. Exp.*, vol. 26, no. 22, pp. 28713–28726, 2018.
- [19] B. Yan et al., "Dirty-data-based alarm prediction in self-optimizing large-scale optical networks," *Opt. Exp.*, vol. 27, no. 8, pp. 10631–10643, 2019.
- [20] H. Zhuang, Y. Zhao, X. Yu, Y. Li, Y. Wang, and J. Zhang, "Machine-learning-based alarm prediction with GANs-based self-optimizing data augmentation in large-scale optical transport networks," in *Proc. 2020 Int. Conf. Comput., Netw. Commun.*, 2020, pp. 294–298.
- [21] C. Y. Zhang, "Research on intelligent prediction and analysis technology for optical network system faults," Beijing Univ. Posts Telecommun., Beijing, China, 2022.
- [22] L. Wang, D. Wang, C. Zhang, S. Liu, and M. Zhang, "Uncertainty analysis for failure prediction in optical transport network using Bayesian neural network," presented at the Opt. Fiber Commun. Conf., Washington, DC, USA, Jun. 6–11, 2021, Paper W6A.45.
- [23] J. N. Yang, "Research on collaborative optical network fault prediction technology based on federated learning," Beijing Univ. Posts Telecommun., Beijing, China, 2023.
- [24] Z. L. Cheng, G. B. Cheng, and Y. M. Cheng, "Process fault prediction model based on CNN-LSTM-LOF," *J. Beijing Univ. Chem. Technol. (Natural Sci. ed.)*, vol. 51, no. 1, pp. 121–127, 2024.
- [25] Y. Li, R. Zhang, P. Zhao, and Y. Wei, "Feature-attended federated LSTM for anomaly detection in the financial Internet of Things," *Appl. Sci.*, vol. 14, no. 13, 2024, Art. no. 5555.
- [26] Z. Ji, W. Tao, and J. Ren, "Boiler furnace temperature and oxygen content prediction based on hybrid CNN, biLSTM, and SE-Net models," *Appl. Intell.*, vol. 54, no. 17–18, 2024, Art. no. 8241.
- [27] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOPS methods for failure management," *ACM Trans. Intell. Syst. Technol.*, vol. 12, no. 6, pp. 1–45, 2021.
- [28] H. Liu et al., "Lightwave fabrics: At-scale optical circuit switching for datacenter and machine learning systems," in *Proc. ACM SIGCOMM 2023 Conf.*, 2023, pp. 499–515.
- [29] X. Yang, Z. An, Q. Pan, L. Yang, D. Lei, and Y. Fan, "Binary optical machine learning: Million-scale physical neural networks with nano neurons," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, 2024, pp. 603–617.
- [30] M. Salani, C. Rottondi, L. Ceré, and M. Tornatore, "Dual-stage planning for elastic optical networks integrating machine-learning-assisted QoT estimation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1293–1307, Jun. 2023.

Zhipeng Zhao received the Ph.D. degree from the College of Intelligence and Computing, Tianjin University, Tianjin, China, in 2019. She is currently a Lecturer with the Henan University of Technology, Zhengzhou, China, and a Postdoctoral Research Fellow with the Chongqing University of Posts and Telecommunications, Chongqing, China. Her research interests include datacenter network, high-performance switch and router, and optical communications.

Lei Guo (Senior Member, IEEE) received the Ph.D. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2006. He is currently a Full Professor with the Chongqing University of Posts and Telecommunications, Chongqing, China. He has authored or coauthored more than 200 technical papers in international journals and conferences. His research interests include communication networks, optical communications, and wireless communications. He is also the Editor for several international journals.

Bin Wu received the Ph.D. degree in electrical and electronic engineering from the University of Hong Kong, Pokfulam, Hong Kong, SAR, China, in 2007. From 2007 to 2012, he was a Postdoctoral Research Fellow with ECE Department, University of Waterloo, Waterloo, ON, Canada. He is currently a Full Professor with the School of Computer Science and Technology, Tianjin University, China. His research interests include computer systems and networking as well as communications.